

Alunas: Naira Juliane Simões e Maria Clara Borges de Medina

Professora: Vera Aparecida de Oliveira

Engenharia de Software, 1º Semestre

30/09/2026

Threads em Sistemas Operacionais

No contexto dos sistemas operacionais, um *thread* (ou fluxo de execução) é a unidade básica de utilização da CPU. Ela é composta por um identificador próprio, um contador de programa (program counter), um conjunto de registradores e uma pilha de execução (stack).

Diferente de um processo que atua como um contêiner isolado para a alocação de recursos de hardware, a thread opera de forma subordinada, inserida no espaço de endereçamento de um processo. Em uma arquitetura multithreaded, múltiplas threads pertencentes a um mesmo processo compartilham a seção de código, a seção de dados e os recursos alocados pelo sistema operacional, como descritores de arquivos e sinais.

Essa estrutura de memória compartilhada permite a execução concorrente de tarefas com um baixo custo de processamento (overhead). Por não exigir o isolamento de memória entre as estruturas internas de um mesmo processo, a troca de contexto (context switch) e a comunicação entre threads tornam-se significativamente mais rápidas e eficientes do que os mecanismos de comunicação entre processos independentes (IPC).

Quando aplicar Threads

Devemos aplicar threads sempre que o programa precisar realizar múltiplas tarefas simultaneamente sem bloquear a execução principal. Os cenários mais comuns incluem:

Interfaces de Usuário (UI): Manter a tela responsiva. Ao baixar um arquivo grande no thread principal, a interface vai congelar. O download deve ocorrer em um thread secundário (background thread).

Servidores Web e APIs: Atender múltiplos clientes ao mesmo tempo. Cada vez que um usuário faz uma requisição ao servidor, um novo thread (ou um pool de threads) é acionada para processar aquela resposta, deixando o servidor livre para receber outras requisições.

Processamento Paralelo: Dividir uma tarefa matemática pesada ou o processamento de grandes volumes de dados em várias fatias, executando-as em diferentes núcleos do processador ao mesmo tempo para reduzir o tempo total.

Exemplo prático

Um exemplo que está diretamente no repositório oficial do Python (CPython) no GitHub, na documentação do módulo `threading`, de forma abreviada:

```
import threading
import time

def crawl(link, delay=3):
    print(f"crawl started for {link}")
    time.sleep(delay)
    print(f"crawl ended for {link}")

links = [
    "https://python.org",
    "https://docs.python.org",
    "https://peps.python.org",
]

threads = []
for link in links:
    t = threading.Thread(
        target=crawl,
        args=(link,),
        kwargs={"delay": 2}
    )
    threads.append(t)

for t in threads:
    t.start()

for t in threads:
    t.join()
```

Esse código cria um thread para cada endereço. As três tarefas começam sem esperar a anterior terminar; depois, `join()` garante que o programa espere todas concluírem. O próprio exemplo usa `sleep()` para representar uma operação de rede bloqueante. Um uso realmente prático seria substituir o `sleep()` por uma requisição HTTP.

Links / Referências:

Entendendo threads e processos. Disponível em: <<https://www.ibm.com/docs/pt-br/aix/7.3.0?topic=programming-understanding-threads-processes>>. Acesso em: 30 set. 2026.

Exemplo Github com Python:

<https://github.com/python/cpython/blob/main/Lib/threading.py?utm>